

Markov Decision Process

Definition, grid world, and the optimal policy

Seung Ho Go

July 12, 2026

Table of contents

- 1 Recap & motivation
- 2 Markov Decision Process
- 3 Grid world as an MDP
- 4 The optimal policy

Recap: from stochastic process to Markov process

A **stochastic process** is a sequence of random variables indexed by time:

$$S_0, S_1, \dots, S_t, S_{t+1}, \dots$$

It has the **Markov property** when

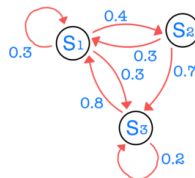
$$P(S_{t+1} | S_t) = P(S_{t+1} | S_0, \dots, S_t).$$

"the future depends only on the present state, not the past."

Such a process is a **Markov process** (S, P) , where P is the **state transition probability**

$$P_{ij} = P(S_{t+1}=s_j | S_t=s_i).$$

Today: add **actions** (decisions) and **rewards** $\Rightarrow$ **Markov Decision Process**.



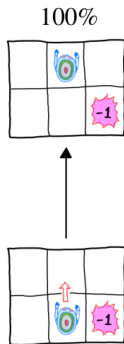
	S ₁	S ₂	S ₃
S ₁	0.3	0.4	0.3
S ₂	0.3	0.0	0.7
S ₃	0.8	0.0	0.2

3-state Markov chain and its row-stochastic transition matrix.

Why “decision”? a grid world that is *stochastic*

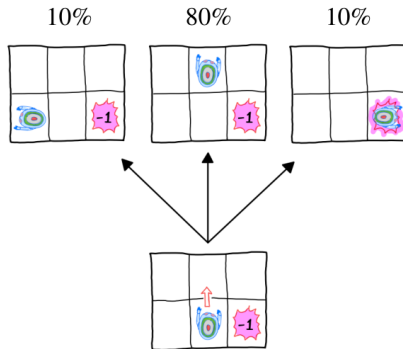
An agent lives in a grid and chooses a direction — but movement is **noisy**.

Deterministic grid world



The model output is fully determined by the present state and action.

Stochastic grid world



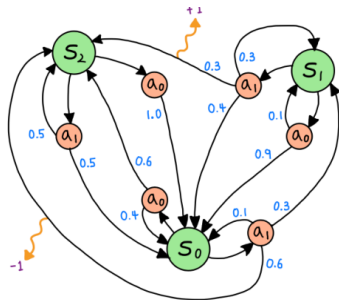
The same state and action will lead to several outputs due to **randomness**.

MDP = Markov process + actions + rewards

An **MDP** is a tuple (S, A, P, R, γ) in which every state has the Markov property.

- S — **state space**
- A — **action space** (new vs. Markov process)
- P — transition probability $P_{ss'}^a$
- R — **reward** function $R_{ss'}^a$
- $\gamma \in [0, 1]$ — discount factor

The action is the “decision”; the reward is the feedback that tells us *which* decisions are good.



An MDP: 3 states S_0, S_1, S_2 , 2 actions a_0, a_1 , rewards $+1, -1$.

Transition probability and reward

With an action in the condition, the transition probability becomes

$$P_{ss'}^a = p(s' | s, a) = P(S_{t+1}=s' | S_t=s, A_t=a).$$

Three common forms of the reward function

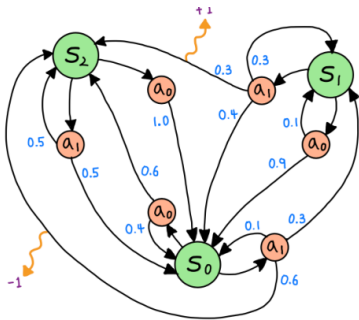
R_s
on entering s

R_s^a
on taking a in s

$R_{ss'}^a$
on the transition $s \xrightarrow{a} s'$

Read $P_{ss'}^a$ left-to-right (engineering style): given **present state** s and **action** a , the probability of **next state** s' .
The bar “|” separates the condition (left) from the outcome (right).

Reading an MDP diagram (worked example)



From state s_0 , take action a_1 . The outgoing probabilities are

$$P_{s_0 s_0}^{a_1} = 0.1$$

$$P_{s_0 s_1}^{a_1} = 0.3 \quad (\text{sum} = 1)$$

$$P_{s_0 s_2}^{a_1} = 0.6$$

They must sum to 1 — the agent goes *somewhere*.

A reward sits on a specific transition, e.g.

$$R_{s_1 s_2}^{a_1} = +1.$$

Model-based vs. model-free

The **environment model** of an MDP is its transition probability P .

Known P —

“known MDP.” We have the full model, so we can **plan** the optimal policy by *Dynamic Programming*.

Unknown P — **model-free**

“unknown MDP.” We cannot see P , so we **learn** from sampled experience — *Reinforcement Learning*.

This single distinction organizes the entire course: plan when you know the world, learn when you don't.

Grid world: states and actions

State set

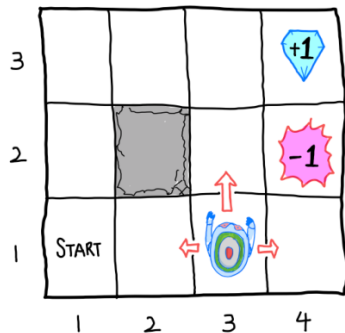
$$S = \{(1, 1), (1, 2), (1, 3), \dots, (4, 2), (4, 3)\}$$

One cell is a wall (unreachable) $\Rightarrow$ **11 states**.

Action set

$$A = \{\text{north, south, east, west}\}$$

The agent starts at START, aims for the +1 cell, and avoids -1.



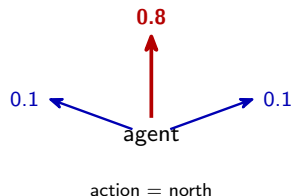
Grid world: stochastic transitions

Intended direction with prob. **0.8**; each perpendicular slip with prob. **0.1**.

$$P_{(3,1)(3,2)}^{\text{north}} = 0.8, \quad P_{(3,1)(2,1)}^{\text{north}} = 0.1, \quad P_{(3,1)(4,1)}^{\text{north}} = 0.1.$$

If a wall blocks the way, the agent **stays in place**.

For this grid there are up to $\sim 9 \times 4 \times 11$ such transition entries
— far too many to reason about by hand.



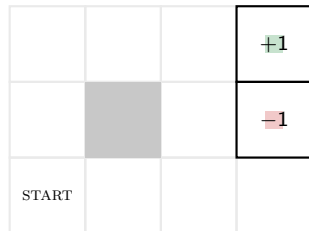
Grid world: two kinds of reward

- **Big reward** at the terminal cells:

$$R_{(3,3)(4,3)}^{\text{east}} = +1, \quad R_{(3,2)(4,2)}^{\text{north}} = -1.$$

- **Small negative reward** c on every other step:

$$R_{ss'}^a = c \quad (c < 0, \text{ e.g. } -0.01).$$

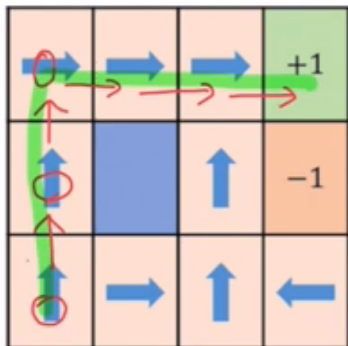


+1 goal · -1 trap · every step costs c

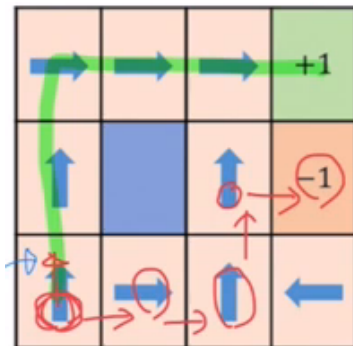
The per-step penalty c discourages wandering: a longer path accumulates more c , so the agent is nudged to **finish efficiently**.

Policy π and episodes: one policy, many outcomes

A **policy** $\pi : S \rightarrow A$ assigns an action to each (non-terminal) state. Running it produces an **episode** (a trajectory). Because transitions are stochastic, *one* policy yields *many* different episodes.



intended path $\rightarrow +1$



a stochastic slip $\rightarrow -1$

a policy fixes the intended action — randomness does the rest.

Same policy (arrows), two episodes:

- intended: $4c + 1 = -0.6$
- detour: -2.2

(with $c = -0.4$; total reward differs run to run)

Evaluating a policy $\Rightarrow$ the optimal policy π_*

Since one policy yields many episodes, we judge it by the **expected total reward**:

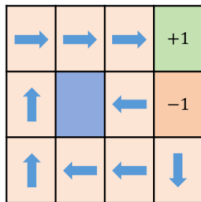
$$\text{value of } \pi = \mathbb{E}_\pi[\text{total reward of an episode}].$$

Optimal policy

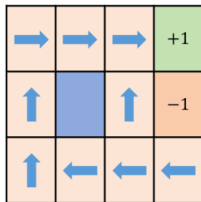
The **optimal policy** π_* is the one that **maximizes** the expected total reward. We mark optimality with a star: π_* .

Finding π_* is the goal of an MDP.

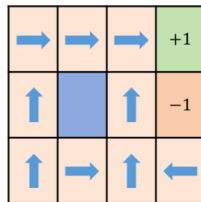
The reward knob: how c reshapes π_*



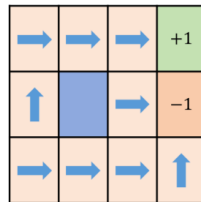
$R_{ss'}^a = -0.01$
 (= c)
 slow but safe



$R_{ss'}^a = -0.03$



$R_{ss'}^a = -0.4$
 appropriate



$R_{ss'}^a = -2$
 finish asap

- $c = -0.01$ — **slow but safe**: detours around -1 even if it takes longer.
 - $c = -0.03$ — still cautious.
 - $c = -0.4$ — **appropriate**: a sensible balance.
 - $c = -2$ — **finish asap**: rushes to the nearest exit, even into -1 .
- different c = different MDP = different optimal policy. **Reward design shapes behavior.***

Takeaways

- 1 **MDP** = (S, A, P, R, γ) : a Markov process plus **actions** and **rewards**.
- 2 The transition probability P is the **environment model**: known $\Rightarrow$ *model-based* (plan by DP); unknown $\Rightarrow$ *model-free* (learn by RL).
- 3 Grid world is **stochastic**: intended 0.8, slip 0.1/0.1.
- 4 A policy π yields **many episodes**; evaluate it by the **expected total reward**.
- 5 The **optimal policy** π_* maximizes it — and **reward design** (c) shapes it.

Next: *how* do we actually compute π_* ? $\rightarrow$ the **Bellman equation** & **Dynamic Programming**.

Thank you

Questions?